

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

The repository files could not be fetched directly from GitHub in this session, so I can reliably base the changes on the `game_engine.py` and `renderer.py` versions you already provided, but I **cannot safely give an exact `main.py` change without seeing its event loop.**

Files that need modification

File	Required change
<code>game/game_engine.py</code>	Add countdown state, freeze gameplay during countdown, and restart/reset logic
<code>game/renderer.py</code>	Display countdown and the new Game Over/restart UI
<code>main.py</code>	Likely needs a small change so <code>R</code> reaches <code>GameEngine.handle_keydown()</code> ; I need its current contents to make this exact

Your `helicopter.py` and `obstacle.py` should **not** be changed.

Important state design

The engine should have three states:



↓ R
COUNTDOWN

During COUNTDOWN :

- `helicopter.update()` is not called.
- obstacles are not spawned or moved.
- `handle_input()` ignores UP/DOWN and `s`.
- countdown displays 3 , 2 , 1 , START! .

On R after Game Over:

```
helicopter → (100, HEIGHT / 2)
vy          → 0
obstacles   → []
distance    → 0
game_over   → False
shield      → False
countdown   → restarted
```

The cleanest implementation uses a countdown timer rather than frame-count assumptions, so the countdown duration is independent of the game's FPS.

Please paste your current `main.py` . I already have the current `game_engine.py` and `renderer.py` from this conversation, so you only need to send `main.py` . Then I can give you the exact complete changes without guessing how your Pygame event loop is wired.

"""

Helicopter Game (Lab Starter)

Run with: `python3 main.py`

Controls: Up/Down arrows to move.

"""

`import pygame`

`from game.game_engine import GameEngine`

Show more ▾

Now that I have `main.py`, the implementation can stay contained to **two files**:

- `game/game_engine.py` — countdown state, freezing gameplay, and complete restart/reset.
- `game/renderer.py` — countdown and Game Over UI.
- `main.py` — **no change required** because it already forwards every `KEYDOWN` event to `engine.handle_keydown()`.

Your existing `helicopter.py` and `obstacle.py` remain unchanged.

1. `game/game_engine.py`

Replace your current `game_engine.py` with this version. It preserves Tasks 1–4 and adds the countdown/restart behavior.

"""

`GameEngine: owns the helicopter and all obstacles.`

```
Handles movement, collisions, distance scoring, shield mechanics,  
start countdown, and restarting after Game Over.  
"""
```

```
import random  
import pygame
```

```
from game.helicopter import Helicopter  
from game.obstacle import Obstacle  
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 90  
GAP_HEIGHT = 150  
WALL_WIDTH = 60  
SCROLL_SPEED = 3
```

```
COUNTDOWN_SECONDS = 4
```

```
class GameEngine:  
    def __init__(self):  
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)  
        self.obstacles = []  
        self.frames_until_spawn = 0  
        self.game_over = False  
        self.distance = 0  
        self.shield_active = False  
  
        self.countdown_active = True  
        self.countdown_start_time = pygame.time.get_ticks()  
  
    def _spawn_obstacle(self):  
        margin = 60  
        gap_y = random.randint(  
            margin + GAP_HEIGHT // 2,
```

```
        HEIGHT - margin - GAP_HEIGHT // 2
    )

    self.obstacles.append(Obstacle(
        x=WIDTH,
        gap_y=gap_y,
        gap_height=GAP_HEIGHT,
        wall_width=WALL_WIDTH,
        screen_height=HEIGHT,
        speed=SCROLL_SPEED,
    ))

def handle_input(self, keys_pressed):
    # Ignore movement input during countdown and after Game Over.
    if self.countdown_active or self.game_over:
        return

    self.helicopter.handle_input(keys_pressed)

def handle_keydown(self, key):
    # Restart after Game Over.
    if self.game_over:
        if key == pygame.K_r:
            self._restart()
        return

    # Shield activation is allowed only during normal gameplay.
    if not self.countdown_active:
        if key == pygame.K_s:
            self.shield_active = True

def _restart(self):
    self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
    self.obstacles = []
    self.frames_until_spawn = 0
```



```
self.game_over = False
self.distance = 0
self.shield_active = False

self.countdown_active = True
self.countdown_start_time = pygame.time.get_ticks()

def _update_countdown(self):
    elapsed = (
        pygame.time.get_ticks() - self.countdown_start_time
    ) / 1000

    if elapsed >= COUNTDOWN_SECONDS:
        self.countdown_active = False

def get_countdown_text(self):
    if not self.countdown_active:
        return None

    elapsed = (
        pygame.time.get_ticks() - self.countdown_start_time
    ) / 1000

    if elapsed < 1:
        return "3"
    elif elapsed < 2:
        return "2"
    elif elapsed < 3:
        return "1"
    else:
        return "START!"

def update(self):
    # Countdown freezes the entire game.
    if self.countdown_active:
```

```
        self._update_countdown()
        return

# Game Over freezes the game.
if self.game_over:
    return

self.helicopter.update(HEIGHT)

self.distance += SCROLL_SPEED

self.frames_until_spawn -= 1
if self.frames_until_spawn <= 0:
    self._spawn_obstacle()
    self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

for obstacle in self.obstacles:
    obstacle.update()

self.obstacles = [
    o for o in self.obstacles
    if not o.is_off_screen()
]

helicopter_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    collision = (
        helicopter_rect.colliderect(obstacle.get_top_rect())
        or helicopter_rect.colliderect(obstacle.get_bottom_rect())
    )

    if collision:
        if self.shield_active:
            self.shield_active = False
```

```
        else:
            self.game_over = True

        break

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.helicopter,
        self.obstacles,
    )

    if self.countdown_active:
        renderer.draw_banner(
            surface,
            font,
            self.get_countdown_text(),
        )
        return

    renderer.draw_text(
        surface,
        font,
        f"Distance: {self.distance}",
        (10, 10),
    )

    if self.shield_active:
        renderer.draw_text(
            surface,
            font,
            "SHIELD ACTIVE",
            (10, 45),
```

```

    )

    if self.game_over:
        renderer.draw_game_over(
            surface,
            font,
            self.distance,
        )

```

Why this works

The countdown starts automatically when `GameEngine()` is created.

During countdown:

```

if self.countdown_active:
    self._update_countdown()

```

Python

Because `update()` returns before moving the helicopter or obstacles, everything remains frozen.

Likewise:

```

if self.countdown_active or self.game_over:

```

Python

prevents UP/DOWN movement during the countdown.

The shield is also prevented from being activated during the countdown.

2. game/renderer.py

Keep your existing constants and drawing functions, but replace `draw_banner()` with this version and add

`draw_game_over()` :

```
def draw_banner(surface, font, text):  
    surf = font.render(text, True, (180, 40, 40))  
    rect = surf.get_rect(  
        center=(  
            surface.get_width() // 2,  
            surface.get_height() // 2  
        )  
    )  
    surface.blit(surf, rect)
```

```
def draw_game_over(surface, font, distance):  
    game_over_surf = font.render(  
        "GAME OVER",  
        True,  
        (180, 40, 40),  
    )
```

```
    distance_surf = font.render(  
        f"Final Distance: {distance}",  
        True,  
        COLOR_TEXT,  
    )
```

```
    restart_surf = font.render(  
        "Press R to Restart",
```

```
        True,  
        COLOR_TEXT,  
    )  
  
    center_x = surface.get_width() // 2  
  
    game_over_rect = game_over_surf.get_rect(  
        center=(center_x, surface.get_height() // 2 - 45)  
    )  
  
    distance_rect = distance_surf.get_rect(  
        center=(center_x, surface.get_height() // 2)  
    )  
  
    restart_rect = restart_surf.get_rect(  
        center=(center_x, surface.get_height() // 2 + 45)  
    )  
  
    surface.blit(game_over_surf, game_over_rect)  
    surface.blit(distance_surf, distance_rect)  
    surface.blit(restart_surf, restart_rect)
```

Your existing `draw_text()` remains unchanged.

3. main.py

No changes are required.

This existing code already does exactly what we need:

Python

Therefore, when the player presses:

- S → `GameEngine.handle_keydown()` activates the shield.
- R after Game Over → `_restart()` resets the game.

Result

Game start

3

then:

2

then:

1

then:

START!

The helicopter and obstacles don't move during these stages.

Normal gameplay

```
Distance: 123  
SHIELD ACTIVE
```

if the shield is active.

After collision without shield

```
GAME OVER  
  
Final Distance: 456  
  
Press R to Restart
```

Press R

Everything resets:

```
helicopter position → initial position  
vertical velocity   → 0  
obstacles           → empty  
distance            → 0  
game_over           → False  
shield_active       → False  
countdown           → starts again
```

Then:

```
3 → 2 → 1 → START! → gameplay
```

Your existing **Task 1 movement/boundaries**, **Task 2 collision detection**, **Task 3 distance scoring**, and **Task 4 one-use shield** remain intact.

WHAT IS CODE FOR render.py

If you mean `game/renderer.py` , use this complete version:

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (140, 200, 230)
COLOR_HELI = (60, 60, 70)
COLOR_OBSACLE = (70, 150, 80)
COLOR_TEXT = (20, 20, 20)

def draw_scene(surface, helicopter, obstacles):
    surface.fill(COLOR_BG)

    for obstacle in obstacles:
        pygame.draw.rect(
            surface,
            COLOR_OBSACLE,
            obstacle.get_top_rect()
        )
```

```
        pygame.draw.rect(
            surface,
            COLOR_OBSACLE,
            obstacle.get_bottom_rect()
        )

    pygame.draw.rect(
        surface,
        COLOR_HELI,
        helicopter.get_rect(),
        border_radius=4
    )

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            surface.get_height() // 2
        )
    )
    surface.blit(surf, rect)

def draw_game_over(surface, font, distance):
    game_over_surf = font.render(
        "GAME OVER",
        True,
        (180, 40, 40)
    )
```

```

distance_surf = font.render(
    f"Final Distance: {distance}",
    True,
    COLOR_TEXT
)

restart_surf = font.render(
    "Press R to Restart",
    True,
    COLOR_TEXT
)

center_x = surface.get_width() // 2
center_y = surface.get_height() // 2

game_over_rect = game_over_surf.get_rect(
    center=(center_x, center_y - 45)
)

Python
    ,      _ ,      _ ,
)

restart_rect = restart_surf.get_rect(
    center=(center_x, center_y + 45)
)

surface.blit(game_over_surf, game_over_rect)
surface.blit(distance_surf, distance_rect)
surface.blit(restart_surf, restart_rect)

```

This supports all four tasks plus the new 3 → 2 → 1 → **START!** countdown and **R to Restart** behavior.

ChatGPT can make mistakes. Check important info.